BrotherMode

A Claude skill that turns the model from a tool that waits for instructions into a colleague that owns outcomes. Not a prompt. A written constitution a session loads before any sizable task, plus the small mechanical toolchain that holds it to that constitution. A prompt tells the model what to do once; the law tells it how to work every time, and the telemetry proves whether it actually did.

16	101	0	1
NUMBERED LAWS	REGRESSION TESTS	NETWORK CALLS, EVER	WRITER PER FILE

WHO IT IS FOR

The solo founder and the individual top performer. A company has a security officer, a data scientist, a project lead, and someone who remembers what you decided last month. Working alone, you have none of them.

BrotherMode gives you the specialists you cannot hire and a memory that survives every interruption. It scales **down** to one person on purpose, not up to an org. Sharing occasionally with a small team is supported; running a multi-team control plane is not what this is for.

THE PHILOSOPHY

> *Prove what you claim. Never perform confidence you have not earned with a passing command run after your last edit.*
>
> SECTION 0, THE FIRST RULE

Most agent setups fail the same way: ceremony on small tasks, no verification on big ones, a killed session that forgets what it was doing, two subagents overwriting each other, and success reported that was never earned.

BrotherMode closes those structurally rather than by hoping the model behaves. Every law exists because something went wrong once and got written down. **Laws live on disk, not in recollection**, so a compaction cannot quietly repeal them.

THE LAWS / 16 SECTIONS

0 Invocation
Classify, assign roles, read memory, map the ground, set the loop, open the state file.

1 Profiles
Build, data, research, content, design, ops each get their own gates.

2 Roles
Only the hats the work needs, named out loud.

3 Decision ladder
Answer, search, ask, inline, one agent, fleet. Stop at the first that suffices.

4 Budgets
Effort tiers declared per brief; spend measured, never estimated.

5 Single writer
One writer per file, ever. Fence before dispatch, never after.

6 Research
Never from memory. Primary sources, cross-checked, cited.

7 Circuit breakers
Two failures revert, three stop and report. No third identical retry.

8 Self-improvement
Rubric before scoring. Findings go to refuters who try to kill them.

9 Context hygiene
Artifacts move as files, never pasted through the orchestrator.

10 Honesty
Bad news first. Push back with your values, then execute your call.

11 Founder gates
Credentials never typed. Releases and destructive acts confirmed.

12 Memory
A structured vault written at every milestone, not at the end.

13 Known mistakes
A ledger of what already went wrong, read before acting.

14 Founder model
Think alike, think against, stay objective.

15 Scoring
Every run closes with a scorecard and an honest Remaining list.

WHAT IT DOES FOR YOU

- **Nothing is silently lost.** A snapshot is taken before every context compaction, including untracked files, into a private local git ref. A killed session resumes from disk.
- **Two agents cannot overwrite each other.** Work is claimed as a record with a real file list, so a collision is computed and refused by name, not noticed later by you.
- **Secrets never reach disk.** Redaction runs at the write boundary, so no code path can forget it, and a gate fails the build when a new write site appears unreviewed.
- **Effort matches the task.** A one-line fix does not get a ten-agent audit, and an audit does not get one hurried pass.
- **Claims are checked before you see them.** Findings go to reviewers whose job is to refute them, on different angles, and the ones that die stay dead.
- **It learns.** Corrections you give become written laws with the reason attached, and a weekly review reverts any change that did not measurably help.

A DAY WITH IT

1. Say `/brothermode` with your task. It classifies the work and says which specialists it is putting on.
2. It reads memory and maps the ground before touching anything: what changed, who else is writing, what failed here before.
3. It states the plan with a **done-check per step**: a command that must pass, not a promise.
4. Work happens behind fences. Independent pieces run at once; nothing merges unverified.
5. It closes with the scorecard, the proof, and the honest list of what is still missing.

GETTING THE MOST FROM IT

- **Correct it out loud.** A correction becomes a written law with its reason, so you never give it twice.
- **Rate the outcome, 1 to 5.** Fifteen seconds. Without it the learning loop is blind and says so.
- **Let it push back.** Disagreement that cites your own stated values is the point, not friction.
- **Read the Remaining list.** That is where the honest gaps live, and it is never allowed to be empty by omission.

WHAT IT IS NOT

- Not enforcement. Most laws are followed by the model, not compelled by the machine. Pair it with CI and protected branches for hard gates.
- Not multi-user. No distributed locking, no org governance. One person, occasionally sharing.
- Not a substitute for reading the diff. It makes verification cheap; it does not make you unnecessary.